

00c - Faden-Karte: Der Weg der Template-ID

Zweck: Den Faden am Sessionanfang in 5 Minuten wiederfinden. Diese Seite ändert sich selten. Sie zeigt **immer dieselbe Geschichte**: Wie die Nummer des Bauplans (Template-ID) durch die Pipeline wandert.

So benutzt du sie am Sessionanfang:

1. Teil A (Landkarte) kurz anschauen – wo bin ich?
2. Teil B von oben nach unten lesen, **jede Zeile von rechts nach links**.
3. Teil C: Claude stellt **eine** Einstiegsfrage, du antwortest mit der Karte daneben.
4. Erst danach: Gedächtnis-Refresh zu dem, was neu ist.

Grundfrage bei jedem Namen: Bauplan oder Haus? Steht `template` im Namen → **Bauplan** (Template, z. B. 100 / 2000). Ein **Haus** ist eine VM (z. B. 1001 `web_server`, 100 `test_vm`).

Begriffe (seit 10.10.): Nur noch **State** bzw. die Datei `terraform.tfstate` – nicht mehr „Grundbuch“.

Zwei Arten von Nummern auf dieser Seite:

Nummer	Wo	Bedeutung
① - ⑤ (im Kreis)	in den Überschriften von Teil B	die fünf Stationen der 2000: an welchem Ort sie gerade ist und was dort mit ihr passiert
1, 2, 3 ... und a, b, c ...	unter den Klammern einer Codezeile	die Teile dieser einen Zeile ; die Tabelle darunter sagt, wer den Teil bestimmt und was er bedeutet

Überschriften **ohne** Kreis-Nummer sind **Zwischenschritte**: Dort passiert etwas, das die 2000 weiterbringt, aber sie wird dort nicht abgelegt.

Zeilennummern: `pipeline.yml`, Stand 10.10.2026 (von Randolph eingefügt).

Teil A - Die Landkarte (wo bin ich?)

Job	Was er tut	Braucht Werte von	Gibt Werte weiter an
JOB 0 <code>detect-changes</code>	schaut, welche Dateien geändert wurden	-	Job 1 (<code>packer</code>)
JOB 1 <code>build-template</code>	Packer baut den Bauplan	Job 0	Job 3 (<code>new_template_id</code>)
JOB 2 <code>security-scan</code>	Checkov prüft Terraform-Code	-	-
JOB 3 <code>deploy-vm</code>	Terraform baut die Häuser	Job 1	Job 5 (IPs), Job 6 (<code>effective_template_id</code>)
JOB 4 <code>ansible-security-scan</code>	Checkov prüft Ansible-Code	-	-
JOB 5 <code>configure-vm</code>	Ansible richtet die Häuser ein	Job 3	-
JOB 6 <code>promote-template</code>	gibt den Bauplan frei	Job 3	-

Die Geschichte unten spielt in **Job 3** und endet in **Job 6**.

Teil B - Die Kette, Zeile für Zeile (Fall 2: Packer lief nicht → 2000)

Wer handelt? Du schreibst den Code (Anweisungen). Terraform, Bash und GitHub führen ihn beim Lauf aus.

Die fünf Stationen auf einen Blick

Station	Ort	Was die Station verkörpert	Wer handelt
①	<code>variables.tf</code>	Herkunft: Hier steht die 2000 als Ersatzwert	du (einmal geschrieben)
②	<code>outputs.tf</code>	Anweisung: „Schreib die 2000 nach dem <code>apply</code> in den State“	du (einmal geschrieben)
③	State (<code>terraform.tfstate</code>)	Ablage: Hier liegt die 2000 nach dem <code>apply</code>	Terraform schreibt (Step 6, Z. 339)
④	Job 3 Step 7, Z. 359	Abholen: Die 2000 wird aus dem State gelesen	Terraform liest, Bash legt ab
⑤	Job 3 Step 7, Z. 380	Weitergeben: Die 2000 geht in die Übergabe-Datei	Bash

Danach: Schauenster im Job-Kopf (Z. 256) → Job 6.

① Station 1 - Herkunft: `variables.tf`

Hier steht die 2000 als Ersatzwert. (vereinfacht in einer Zeile, in der Datei mehrzeilig)

```
variable "test_template_id" { default = 2000 }
```

1 2 3 4

Nr	Teil	Wer bestimmt das?	Bedeutung
1	<code>variable</code>	festes Wort von Terraform	„Ich melde eine Variable an“
2	<code>"test_template_id"</code>	du , frei gewählt	Name der Variable
3	<code>default</code>	festes Wort von Terraform	Ersatzwert, wenn kein Zettel da ist
4	<code>2000</code>	du	der Ersatzwert selbst

Zwischenschritt - Job 3 Step 6, Z. 336/337: Zettel hinlegen (oder nicht)

Entscheidet, ob Terraform die 2000 nimmt oder einen anderen Wert.

```
export TF_VAR_test_template_id="{{ steps.effective-template.outputs.template_id }}"
```

1 2 3 4 5

Nr	Teil	Wer bestimmt das?	Bedeutung
1	<code>export</code>	festes Wort von Bash	„Leg einen Zettel in die Umgebung“
2	<code>TF_VAR_</code>	fester Anfang, von Terraform vorgegeben	„Dieser Zettel ist für Terraform“
3	<code>test_template_id</code>	du , muss exakt so heißen wie in <code>variables.tf</code>	für welche Variable der Zettel ist
4	<code>=</code>	Bash	Zuweisung, ohne Leerzeichen davor oder danach
5	<code>{{ ... }}</code>	GitHub setzt es vor dem Start ein	der Wert: Auftrag aus Step 2

Nr. 5 genauer zerlegt:

```
{{ steps.effective-template.outputs.template_id }}
```

a b c d

Nr	Teil	Wer bestimmt das?	Bedeutung
a	steps	festes Wort von GitHub	„ein Step in diesem Job“
b	effective-template	du : die id: von Step 2 (Z. 274)	welcher Step
c	outputs	festes Wort von GitHub	„dessen Übergabe-Datei“
d	template_id	du : der Name links vom = in Z. 284	welcher Eintrag darin

Fall 2: Der Auftrag ist leer → `if` in Z. 336 ist falsch → **Z. 337 wird übersprungen** → **kein Zettel** → Terraform nimmt die 2000 aus ①.

Zwischenschritt - Job 3 Step 6, Z. 339: Terraform baut und schreibt

Hier wird die 2000 benutzt (Klonen) und nach Anweisung ② in den State ③ geschrieben.

```

terraform apply -auto-approve
  1         2         3

```

Nr	Teil	Wer bestimmt das?	Bedeutung
1	terraform	das Programm auf dem Runner	Terraform wird gestartet
2	apply	festes Wort von Terraform	„Bauen und danach in den State schreiben“
3	-auto-approve	Schalter von Terraform	ohne Rückfrage „Wirklich?“

Was Terraform bei `apply` tut, der Reihe nach:

- liest den **Code** (alle `.tf`-Dateien) → was es geben **soll**
- setzt den **Lock** auf den State
- liest den **State** → was es **weiß**
- fragt Proxmox: Stehen die VMs aus dem State noch?
- vergleicht Code ↔ State → was fehlt?
- lässt Proxmox bauen; Proxmox meldet zurück (Nummer, ...)
- schreibt in den State**, erst **nachdem** die VM steht (Quittung)
- entfernt den Lock

② Station 2 - Anweisung: `outputs.tf`

Hier steht, dass die 2000 nach dem `apply` in den State soll. (Du hast das vorher geschrieben; ausgeführt wird es bei Z. 339.)

```

output "used_test_template_id" { value = var.test_template_id }
  1         2                 3      4      5

```

Nr	Teil	Wer bestimmt das?	Bedeutung
1	output	festes Wort von Terraform	„Nach dem <code>apply</code> unter diesem Namen in den State schreiben“
2	"used_test_template_id"	du , frei gewählt	Name des Eintrags im State
3	value	festes Wort von Terraform	welcher Wert eingetragen wird
4	var	festes Wort von Terraform	„aus einer Variable“
5	test_template_id	du : Name aus <code>variables.tf</code> (①)	aus welcher Variable

Du schreibst diese Zeile einmal. **Terraform** führt sie bei jedem `apply` aus.

③ Station 3 - Ablage: der State (`terraform.tfstate`)

Hier liegt die 2000 nach dem `apply`.

Name	Wert
used_test_template_id	2000

(Am 09.10. im echten State auf soar-dev-server per grep gesehen.)

④ Station 4 - Abholen: Job 3 Step 7, Z. 359

Hier wird die 2000 aus dem State gelesen.

```
USED_TEMPLATE_ID=$(terraform output -raw used_test_template_id 2>/dev/null)
```

1 2 3 4 5 6 7

Nr	Teil	Wer bestimmt das?	Bedeutung
1	USED_TEMPLATE_ID	du , frei gewählt (GROSS)	Bash-Variable, entsteht hier
2	=\$(...)	Bash	„Führ den Befehl in der Klammer aus, leg das Ergebnis links ab“
3	terraform	das Programm	Terraform wird (zum zweiten Mal) gestartet
4	output	festes Wort von Terraform	„Lies einen Eintrag aus dem State“ - liest nur den State
5	-raw	Schalter von Terraform	nur der nackte Wert, ohne Anführungszeichen
6	used_test_template_id	du : Name aus outputs.tf (@, klein)	welcher Eintrag
7	2>/dev/null	Bash	Fehlermeldungen (Kanal 2) in die Mülltonne

Zwei fast gleiche Namen: **1 GROSS** = Bash-Variable, **6 klein** = Name im State.

⑤ Station 5 - Weitergeben: Job 3 Step 7, Z. 380

Hier geht die 2000 in die Übergabe-Datei.

```
echo "used_template_id=$USED_TEMPLATE_ID" >> $GITHUB_OUTPUT
```

1 2 3 4 5 6

Nr	Teil	Wer bestimmt das?	Bedeutung
1	echo	festes Wort von Bash	„Gib aus“
2	used_template_id	du , frei gewählt	Name für die Übergabe
3	=	Format von GitHub	trennt Name und Wert
4	\$USED_TEMPLATE_ID	Bash setzt den Wert ein	2000 aus ④
5	>>	Bash	an eine Datei anhängen
6	\$GITHUB_OUTPUT	GitHub gibt den Dateipfad vor	die Übergabe-Datei dieses Steps

(In dieser Darstellung stehen zur besseren Lesbarkeit zwei Leerzeichen nach echo; in der Datei ist es eins.)

Zwischenschritt - Job 3 Job-Kopf, Z. 256: ins Schaufenster stellen

Macht die 2000 aus ⑤ für Job 6 sichtbar.

```
effective_template_id: ${ steps.get-ip.outputs.used_template_id }
```

1 2 3 4 5 6 7

Nr	Teil	Wer bestimmt das?	Bedeutung
1	<code>effective_template_id</code>	du , frei gewählt	Name im Schaufenster des Jobs
2	<code>:</code>	YAML	trennt Name und Wert
3	<code>{{{ ... }}}</code>	GitHub	wird am Job-Ende eingesetzt
4	<code>steps</code>	festes Wort von GitHub	„ein Step in diesem Job“
5	<code>get-ip</code>	du : die <code>id</code> : von Step 7 (Z. 351)	welcher Step
6	<code>outputs</code>	festes Wort von GitHub	„dessen Übergabe-Datei“
7	<code>used_template_id</code>	du : Name aus ⑤ Nr. 2	welcher Eintrag

Job 6 liest es mit `needs.deploy-vm.outputs.effective_template_id` (Z. 616, 625, 629).

Die ganze Kette in einer Zeile

① `variables.tf` (2000) → kein Zettel → `apply` → ② Anweisung → ③ State → ④ abholen → ⑤ weitergeben
→ Schaufenster → Job 6

Teil C - Einstiegsfragen (Claude stellt EINE davon)

1. Lies die Zeile von ④ von rechts nach links: **Wer tut was?**
2. Wer **schreibt** die 2000 in den State (③), und bei welchem Befehl?
3. Warum gibt es in Fall 2 keinen Zettel? (Welche Zeile wird übersprungen?)
4. `outputs.tf` (②): Welche Teile hast **du** gewählt, welche sind feste Wörter?
5. Bauplan oder Haus: 100, 2000, 1001 – und wer macht sie?
6. (neu 10.10.) Was tut Terraform bei `apply` **vor** und **nach** dem Bauen mit dem State?
7. (neu 10.10.) Nenne zu jeder Station ①–⑤ in einem Wort, was sie verkörpert.

Pflege: Nur ändern, wenn sich die Pipeline an diesen Stellen ändert (z. B. nach der geplanten Umbenennung von `effective_template_id`) oder ein grundlegend neuer Faden dazukommt (z. B. Phase 3 – dann eigener Teil D). Letzte Änderung: 10.10.2026 – Klammern mit Nummern + Tabellen, Stationen ①–⑤ mit Bedeutung, Zeilennummern nach `pipeline.yml` vom 10.10., „Grundbuch“ → State.